

Программная инженерия

Лекция 12: CI и вайб-кодинг

Денисов Артём Владимирович

Уральский Федеральный Университет

2026

Определение

CI/CD — это конвейер проверки и доставки кода.

CI — Continuous Integration

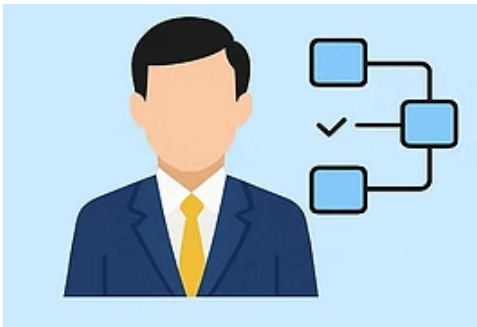
«Можно ли это изменение безопасно объединить с основным кодом?»

CD — Continuous Delivery

«Можно ли это изменение безопасно доставить пользователям?»

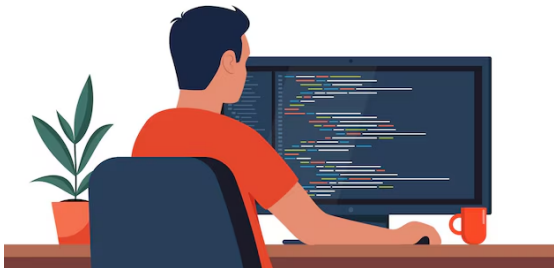
Основные шаги CI/CD

Процесс разработки



Проджект менеджер создаёт задачу

Процесс разработки



Разработчик пишет код

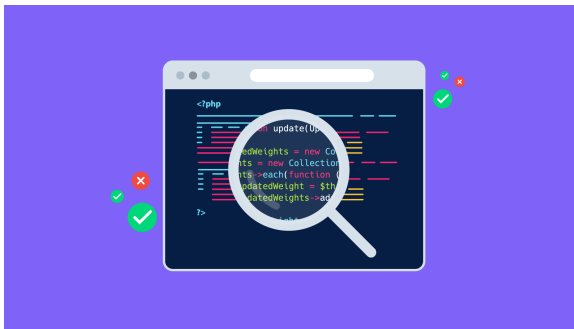
- Код тестируется локально? Или нет...
- Unit тесты



Код отправляется в удалённый репозиторий

- Начинает работу CI/CD пайплайн

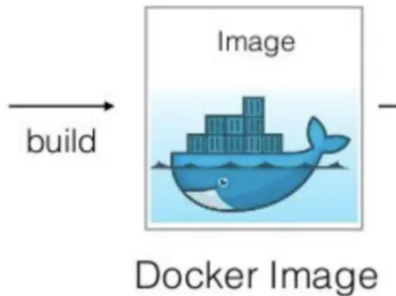
Шаги CI



До сборки. Статическая проверка кода

- Линтеры, форматтеры
- Быстро, не отлавливаются ошибки выполнения

Шаги CI



Сборка проекта

- Проект компилируется, собирается
- Создаётся исполняемый файл (часто Docker образ)

В коде могут быть баги. Что делать?



Тестирование

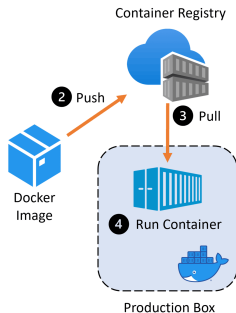
- Unit тесты
- Интеграционные тесты
- Security тесты
- ...



Релиз

- Финальный образ отправляется в registry

Шаги CD



Развёртывание

- Образ разворачивается на staging
- E2E тесты на staging
- Деплой на production

CI vs CD

CI заканчивается там, где мы получили проверенное изменение

CD начинается там, где это изменение начинают доставлять в окружение:
staging, production

Что происходит после доставки

- Ручное тестирование
- Мониторинг состояния системы
- Откат при проблемах

Что изменилось с приходом AI агентов?

The Impact of AI on Developer Productivity: Evidence from GitHub Copilot

Abstract

Generative AI tools hold promise to increase human productivity. This paper presents results from a controlled experiment with GitHub Copilot, an AI pair programmer. Recruited software developers were asked to implement an HTTP server in JavaScript as quickly as possible. The treatment group, with access to the AI pair programmer, completed the task 55.8% faster than the control group. Observed heterogenous effects show promise for AI pair programmers to help people transition into software development careers.

- Исследование 2023 года
- <https://arxiv.org/pdf/2302.06590>

Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity

Abstract

disallow usage of early-2025 AI tools. When AI tools are allowed, developers primarily use Cursor Pro, a popular code editor, and Claude 3.5/3.7 Sonnet. Before starting tasks, developers forecast that allowing AI will reduce completion time by 24%. After completing the study, developers estimate that allowing AI reduced completion time by 20%. Surprisingly, we find that allowing AI actually *increases* completion time by 19%—AI tooling slowed developers down. This slowdown also contradicts predictions from experts in economics (39% shorter) and ML (38% shorter). To understand this result, we collect and evaluate evi-

- Исследование 2025 года
- <https://arxiv.org/pdf/2507.09089>

Вайб-кодинг увеличивает нагрузку на review

*«Low-experience vibe coders produce **2.15**× more commits, **1.47**× more changed files and **4.52**× more review comments»*

- Исследование 2026 года
- <https://arxiv.org/abs/2602.23905>

Код от AI содержит больше уязвимостей

*«Analysis identified 733 AI generated snippets, revealing a high likelihood of security weaknesses, with **29.5%** of Python and **24.2%** of JavaScript snippets affected»*

- Исследование 2025 года
- <https://arxiv.org/pdf/2310.02059>

*«PRs enhanced by Copilot for PRs require **less review time** and have a **higher likelihood of being merged**»*

- Исследование 2024 года
- <https://arxiv.org/abs/2402.08967>

Процесс замедляется в другом месте

Было

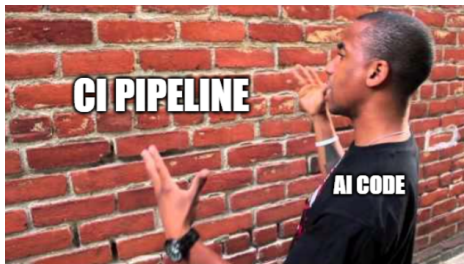
Бутылочное горлышко — написание кода

Стало

Бутылочное горлышко — ревью кода

Переписать код быстрее чем провести review

Роль CI при вайб-кодинге



CI пайплайн должен отгораживать неподходящий код от ручного review

Чем сложнее пройти CI пайплайн, тем качественнее код в репозитории

Как настроить CI пайплайн для
вайб-кодинга?

Ограничение на количество изменений

- Большой PR (больше 200 строчек изменено) - сразу отклоняется
- Маленькие PR проще проходят review, содержат меньше багов
- Проще откат

Строгий статический анализ кода

- Линтеры
- Проверка утечки секретов
- Code style
- ...

Использовать альтернативы метрики coverage

- Нейросети легко пишут тесты - заглушки на прохождение code coverage
- Стоит использовать более сложные метрики тестирования кода

Обязательно ручное review изменений

- Минимум 2 review для одобрения PR + AI review
- Долго, но более безопасно от багов

Большой CI - тоже плохо

Проблема

Длинный CI runtime увеличивает задержку, тратит ресурсы сервера.

Быстрое решение

Добавлять в CI пайплайн только необходимые события (push в dev/main ветку)



SonarQube - это

платформа для автоматизированного code review и статического анализа кода.

SonarQube показывает качество кода

- Баги - некорректное выполнение задачи
- Уязвимости - части кода, открытые для атак
- Сложность кода
- Дубликаты
- Покрытие тестами
- Удобство расширения (code smells)

Чем меньше обнаружено этих проблем, тем выше считается качество кода

SonarQube server

- Правила (инструкции best practices, определяют хороший и плохой код)
- База данных (хранит результаты анализа)
- Веб-интерфейс
- Elasticsearch (поиск нужных данных в БД)

SonarQube scanner

- Проверяет исходный код и отправляет отчет на SonarQube server

Как работает SonarQube



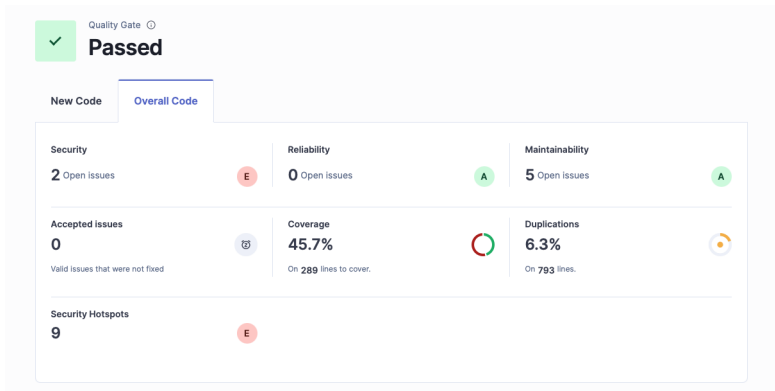
Sonar scanner клиент и SonarQube сервер

Когда запускать SonarQube в CI?

SonarQube должен запускаться после того, как проект уже собран и coverage-отчёты сгенерированы

SonarQube сам не генерирует отчеты покрытия и тесты. Он импортирует и отображает их

SonarQube как Quality Gate



SonarQube может определять когда код достаточно качественный для прохождения в релиз

Источники информации и изображений

1. <https://www.projectmanagertemplate.com/post/project-coordinator-vs-project-manager-key-differences>
2. <https://deepakatanalytics.hashnode.dev/the-ultimate-guide-to-git-and-github>
3. https://www.magnific.com/premium-vector/young-man-writing-code-desktop-computer-software-developer-work-programming_78506500.htm
4. <https://tsh.io/blog/php-static-code-analysis>
5. <https://medium.com/@saurabh.singh0829/how-to-build-application-inside-and-outside-docker-dockerfile-structure-and-commands-f542b58cd830>
6. <https://codestar.vn/product/khoa-hoc-automation-test/>
7. <https://www.youtube.com/watch?v=pFKwmEdwZZQ>
8. <https://juliocasal.com/blog/Docker-Tutorial-For-Dotnet-Developers>
9. https://www.youtube.com/watch?v=E5hM0GeBT-o&list=PLxzKY3wu0_FL3TzBnBeBoIMoRkXmYe3VB

Photographs used under public domain or fair usage. No copyright infringement intended. If used accidentally under wrong license please contact to be removed.

Программная инженерия

Лекция 12: CI и вайб-кодинг

Денисов Артём Владимирович

Уральский Федеральный Университет

2026